

Portfolio of Evidence (POE) – Part 2

Building the Mobile Application

Chef's Menu Manager – Christoffel's Kitchen

Submission detail	Information
Student name	[Enter your full name]
Student number	[Enter your student number]
Lecturer	[Enter lecturer name]
Submission date	6 October 2026
GitHub repository	[Paste your GitHub repository URL after uploading]
YouTube demonstration	[Paste your unlisted YouTube URL after recording]

Submission note: Replace every square-bracket placeholder before handing in. The report is designed to accompany the submitted App.js source file and the GitHub repository.

1. Introduction

Christoffel's Kitchen needs a practical way to keep its menu information organised. The original scenario used paper notes, which can become difficult to update, share and keep consistent. For this part of the portfolio, I turned the proposed solution into a working React Native mobile application called Chef's Menu Manager.

The application focuses on the everyday task the chef actually needs to complete: capturing a dish and seeing it immediately appear on the current menu. A dish has a name, description, course and price. The app also gives clear feedback when information is missing or invalid, confirms successful additions, and explains what to do when the menu is still empty.

I deliberately kept the scope manageable. Rather than adding features that were not necessary for the brief, I concentrated on making the core workflow straightforward: enter the information, choose the course, add the dish, and review the resulting menu.

2. Application Purpose and Main Features

- Capture a dish name, description, course and price.
- Choose one of three courses: Starter, Main Course or Dessert.
- Validate required fields before a dish can be added.
- Display saved dishes as readable menu cards.
- Place the newest dish at the top of the list.
- Show a running count of menu items.
- Provide an empty-state message when no dishes have been added.
- Provide specific error feedback and a temporary success message.
- Keep the screen usable when the mobile keyboard is open.

3. User Interface and Design Decisions

3.1 Layout and visual hierarchy

I designed the screen around the chef's main task rather than around the technical structure of the code. The title identifies the application, the form appears before the menu, and the most important action—adding a dish—is visually prominent. The menu then appears underneath, so the result of the action is easy to find.

- The header uses a clear application title and the restaurant name, so the purpose of the screen is immediately obvious.
- A light background and white cards separate the form and menu content without making the screen feel crowded.
- Dark text on light surfaces improves readability and creates a clear visual hierarchy.
- The three course options are visible at the same time. The selected option changes appearance, making the current choice easy to recognise.

- The main action button uses the same accent colour as the selected course and prices, creating consistency across the interface.
- Spacing, rounded cards and short labels keep the interface approachable on a small screen.

These decisions are consistent with the general usability principle of making interfaces clear, consistent and aligned with the user's expectations (Nielsen, 1994).

3.2 React Native components

Component	Purpose in the application
View and Text	Provide the structure, headings, labels, messages and menu-card content.
TextInput	Capture the dish name, description and price.
TouchableOpacity	Provide the course-selection controls and the Add Item to Menu action.
ScrollView	Allow the form and growing menu to remain accessible on a smaller screen.
KeyboardAvoidingView	Reduce the risk of the mobile keyboard covering the active form area.
SafeAreaView and StatusBar	Keep the interface within the usable device area and maintain a consistent status-bar appearance.
StyleSheet	Keep presentation rules grouped together so the interface is easier to maintain.

3.3 Why I used buttons for course selection

There are only three possible courses, so a drop-down control would add an extra interaction without solving a real problem. Showing all three choices at once lets the chef select a course immediately and makes the selected value visible. This is especially useful when the application is being used quickly during normal kitchen work.

4. How the Application Works

4.1 State management

The application is implemented as a functional React component. React's useState hook stores values that change while the app is running (React, 2024). The state is separated conceptually into three groups:

State	Role
dishName, description, course, price	Store the values currently being entered in the form.
errorMessage, successMessage	Control the validation and confirmation feedback shown to the user.
menuItems	Store the dishes that have been added during the current app session.

4.2 Adding a dish

- When Add Item to Menu is pressed, previous feedback messages are cleared.
- The text values are trimmed so that spaces alone cannot be treated as meaningful input.
- The app checks that the dish name and description are present and that a price has been entered.
- The price is checked against a numeric pattern that permits a whole number or a value with one or two decimal places. This is stricter than simply using `parseFloat` and prevents input such as 12abc from being accepted.
- The price is converted to a number and must be greater than zero.
- A new dish object is created with a generated id, cleaned text, selected course and price formatted to two decimal places.
- The new item is placed at the beginning of the menu array, so the latest addition appears first.
- The form is cleared and the course returns to Starter, reducing the chance that information from the previous dish is accidentally reused.
- A success message confirms the addition and is removed after three seconds. The timer is cleared when necessary so repeated additions do not create competing timers.

4.3 Displaying the menu

When `menuItems` is empty, the application displays a short empty-state message. Once items exist, the array is mapped into menu cards. Each card shows the dish name, formatted price, course and description. The menu count is calculated directly from `menuItems.length`, so it changes automatically as items are added.

For this small application, a `ScrollView` with `map()` is a reasonable choice because the form and menu are part of the same scrolling screen. If the application were later expanded to support hundreds or thousands of dishes, a virtualised list such as `FlatList` would be a better fit.

5. Requirements Checklist

5.1 Build the user interface

Requirement	Evidence in the application
Clear application title	Chef's Menu Manager and Christoffel's Kitchen are displayed at the top.
Appropriate React Native components	<code>TextInput</code> , <code>TouchableOpacity</code> , <code>View</code> , <code>ScrollView</code> , <code>KeyboardAvoidingView</code> , <code>SafeAreaView</code> and <code>Text</code> are used.
Sensible layout and readable text	The form is grouped in a card, fields are labelled, spacing is consistent and high-contrast text is used.

5.2 Capture menu information

Requirement	Evidence in the application
Dish name	Single-line <code>TextInput</code> with required-field validation.
Description	Multi-line <code>TextInput</code> with required-field validation.

Course	Starter, Main Course and Dessert are selectable using touch controls.
Price	Decimal keyboard is provided and the value is validated before saving.

5.3 Display menu items

Requirement	Evidence in the application
Show captured items	Each item is displayed in a card containing its name, price, course and description.
Multiple items	menuItems stores multiple dishes and the interface renders each one.
Automatic update	Updating menuItems causes React to render the new list immediately.
Newest item first	Each new dish is inserted at the start of the array.

5.4 Improve user experience

Requirement	Evidence in the application
Input validation	Blank required fields, malformed prices and non-positive prices are rejected with specific feedback.
Success feedback	A confirmation message appears after a successful addition and disappears after three seconds.
Empty-state feedback	A clear message explains what to do when the menu has no items.
Additional polish	Keyboard avoidance, form reset, course reset, menu count, accessibility labels and consistent visual feedback are included.

6. Testing and Verification

The following test cases should be demonstrated on the running application and, where required by the lecturer, captured as screenshots or shown in the video. The table records the expected behaviour and gives a clear checklist for final verification. I have not treated a source-code review as proof that the app was successfully run on a physical device; the final submission should include the student's own runtime evidence.

#	Test input/action	Expected result
1	Open the app before adding anything.	Empty-state message is visible.
2	Press Add Item to Menu with all fields empty.	A dish-name validation message appears and no item is added.
3	Enter a name but leave the description empty.	A description validation message appears.
4	Enter name and description but leave price empty.	A price validation message appears.
5	Enter price abc.	A valid-price message appears and no item is added.

6	Enter price -5.	A valid-price message appears and no item is added.
7	Enter price 0.	A valid-price message appears and no item is added.
8	Enter price 12abc.	A valid-price message appears; the value is rejected.
9	Enter a name containing only spaces.	The name is treated as empty and rejected.
10	Add Garlic Bread, description, Starter, 5.50.	Item is added and a success message appears.
11	Add Grilled Salmon, description, Main Course, 18.	Price displays as 18.00 and the new item appears above the previous item.
12	Add Chocolate Fondant, description, Dessert, 7.	Third item appears and the count becomes 3.
13	After a successful addition, inspect the form.	Name, description and price are cleared and course returns to Starter.
14	Add two dishes quickly one after another.	The latest success message remains visible for its intended period rather than being cleared by an older timer.

6.1 What I learned from testing

Validation was one of the most useful areas to think through carefully. A simple empty-field check is not enough for a price field. The final implementation therefore checks the format and the numerical value before creating the menu item. I also considered the behaviour of the temporary success message. Because the user may add dishes close together, the previous timer is cleared before a new one is created.

This testing also helped me think beyond whether the code 'works' and focus on what the chef experiences. Clear errors, a visible selected course, a reset form and an immediate menu update all reduce uncertainty during use.

7. Challenges and Problem-Solving

The biggest challenge was making validation specific enough to prevent bad data without making the form frustrating. I initially relied on `parseFloat`, but that function can interpret the numeric beginning of a string such as 12abc as 12. The revised implementation uses a simple price pattern first and then checks that the resulting number is finite and greater than zero.

Another issue was the temporary success message. A message that disappears after a fixed delay sounds simple, but repeated actions can create overlapping timers. Keeping the timer in a ref and clearing the previous timer before creating another makes the feedback behaviour more predictable.

I also paid attention to the small details that are easy to overlook in a short application. The empty state tells the user what to do next, the form resets after a successful save, and accessibility labels provide additional context for assistive technologies. These changes do not make the feature list dramatically larger, but they make the application feel more complete.

8. Future Improvements

- Persist menu data using local storage so dishes survive when the app is closed.
- Add edit and delete actions for existing dishes.
- Group or filter menu items by course.
- Support a configurable currency or local pricing format.
- Add search for larger menus.
- Move the menu data into a more scalable data layer if the application grows.
- Add automated tests for validation and menu-state behaviour.

These are deliberately treated as future improvements rather than claimed features. They are outside the current implementation and should not be presented as completed functionality.

9. How to Run the Application

The application is intended to run as a React Native/Expo project. The exact project setup should match the student's development environment and the module instructions. The submitted repository should contain the project configuration files as well as App.js; submitting only App.js is not sufficient to recreate the project.

- Install Node.js and the required Expo tooling for the project.
- Create or open the Expo project used for the assignment.
- Place the submitted App.js in the project root and ensure it matches the version committed to GitHub.
- Install the project's dependencies using the package configuration included in the repository.
- Run the project with the Expo development command used by the project.
- Open the application using Expo Go on a compatible phone or an available Android/iOS emulator.
- Complete the testing checklist in Section 6 and capture the required evidence.

10. Video Demonstration Plan

Time	What to demonstrate
0:00–0:30	Introduce yourself, the scenario and the purpose of Chef's Menu Manager.
0:30–1:00	Show the main screen, form fields, course controls, Add button and empty-state message.
1:00–2:00	Demonstrate validation using blank input and invalid prices, including 12abc.

2:00–3:30	Add a Starter, Main Course and Dessert. Show the success feedback, count and newest-first ordering.
3:30–4:30	Briefly show the relevant code: state variables, validation, handleAddDish and menu rendering.
4:30–5:00	Summarise what the app achieves and show the GitHub repository/report location.

Recording tip: keep the demonstration focused on evidence. The viewer should be able to see the application working rather than only hearing that it works. Avoid showing passwords, personal account details or unrelated browser tabs.

11. Conclusion

Chef's Menu Manager provides a focused digital replacement for the paper-based menu workflow described in the scenario. The application captures the required dish information, gives the chef clear choices for course selection, validates input, displays multiple menu items and provides feedback after user actions.

The implementation also shows the reasoning behind the main technical decisions rather than treating the code as a collection of controls. React state drives the form and menu, validation protects the data being entered, and the interface is structured around the chef's workflow. The result is intentionally small, but it provides a solid and understandable foundation that could be extended with persistence, editing, searching and other restaurant-management features.

12. Final Submission Checklist

- Enter your real name, student number and lecturer name on the cover page.
- Replace the GitHub placeholder with the final repository URL.
- Replace the YouTube placeholder with the final unlisted video URL, if the assignment requires a video.
- Run the application and complete every test in Section 6.
- Capture screenshots of the working application if screenshots are required by the brief.
- Ensure the GitHub repository contains the complete runnable project, not only App.js.
- Check that App.js in GitHub is the same version demonstrated in the video and described in this report.
- Add a clear README.md with setup, run instructions and feature summary.
- Do not commit passwords, API keys, personal access tokens, node_modules, or other unnecessary generated files.
- Open the final GitHub repository in a private/incognito browser window or another account to confirm that the lecturer can access it.
- Open the submitted report after exporting to PDF and check page breaks, tables, hyperlinks and placeholders.

- Confirm the final upload/submission contains every item required by the lecturer's submission portal.

References

Expo. 2024. Create a project. Available at: <https://docs.expo.dev/get-started/create-a-project/> [Accessed 6 October 2026].

Meta Platforms, Inc. 2024a. Core components and native components. React Native documentation. Available at: <https://reactnative.dev/docs/intro-react-native-components> [Accessed 6 October 2026].

Meta Platforms, Inc. 2024b. TextInput. React Native documentation. Available at: <https://reactnative.dev/docs/textinput> [Accessed 6 October 2026].

Meta Platforms, Inc. 2024c. TouchableOpacity. React Native documentation. Available at: <https://reactnative.dev/docs/touchableopacity> [Accessed 6 October 2026].

Meta Platforms, Inc. 2024d. KeyboardAvoidingView. React Native documentation. Available at: <https://reactnative.dev/docs/keyboardavoidingview> [Accessed 6 October 2026].

Meta Platforms, Inc. 2024e. StyleSheet. React Native documentation. Available at: <https://reactnative.dev/docs/stylesheet> [Accessed 6 October 2026].

Nielsen, J. 1994. 10 usability heuristics for user interface design. Nielsen Norman Group. Available at: <https://www.nngroup.com/articles/ten-usability-heuristics/> [Accessed 6 October 2026].

React. 2024. useState. React documentation. Available at: <https://react.dev/reference/react/useState> [Accessed 6 October 2026].

Appendix A – Source Code

The complete source code is submitted as App.js and should also be committed to the GitHub repository. The report intentionally does not reproduce every line of the source code a second time; instead, this appendix identifies the source file and the key implementation areas that should be checked against it.

Source-code area	What it demonstrates
State declarations	Form, feedback and menu state are maintained with React hooks.
handleAddDish	Validation, object creation, state update, form reset and success feedback.
Course selector	The three course options are rendered from the COURSES array.
Menu rendering	Empty-state logic and mapping of menuItems into cards.
Stylesheet	Consistent layout, typography, spacing, feedback boxes and menu cards.

Accessibility	Labels and roles are provided for key interactive controls and feedback.
---------------	--

Appendix evidence note: Before submission, confirm that the App.js file uploaded to GitHub is the revised version used for the final demonstration.